

QUEUE: Basics & Concepts

A **Queue** is a **linear data structure** that follows the **FIFO (First In, First Out)** principle. Like a real-world queue (e.g., line at a ticket counter), the first person who enters the queue is the first to leave.

Intuition

- Think of people **standing in a line**.
- **Newcomers join at the rear** (enqueue).
- **Service is given from the front** (dequeue).
- Hence, the **first person to enter is served first**.

Key Operations

Operation	Description
<code>enqueue(x)</code>	Add element <code>x</code> to the rear
<code>dequeue()</code>	Remove and return element at front
<code>front()</code>	View the front element
<code>isEmpty()</code>	Check if queue is empty
<code>size()</code>	Number of elements in queue

Real-World Applications

Application	Use Case Example
OS scheduling	CPU task scheduling
Web server	Request handling
Call center	Handling calls in order
Print queue	Printing documents in order

Application	Use Case Example
BFS traversal in graphs	Breadth-First Search
Caching	FIFO Cache Eviction
Buffer/Stream	Data streams, network packets

Types of Queues

1. **Simple Queue** – Basic FIFO queue
2. **Circular Queue** – Last connects to first (avoids memory wastage)
3. **Priority Queue** – Elements served based on priority
4. **Deque (Double Ended Queue)** – Insertion/deletion at both ends

Implementation

✓ Python (Using list and `collections.deque`)

```
from collections import deque

# Basic Queue using deque
q = deque()

q.append(10)      # enqueue
q.append(20)
print(q.popleft()) # dequeue → 10
print(q[0])       # front → 20
print(len(q))     # size → 1
print(not q)      # isEmpty → False
```

✓ Java (Using Queue Interface and LinkedList)

```
import java.util.*;

public class QueueDemo {
```

```

public static void main(String[] args) {
    Queue<Integer> q = new LinkedList<>();

    q.offer(10);           // enqueue
    q.offer(20);
    System.out.println(q.poll()); // dequeue → 10
    System.out.println(q.peek()); // front → 20
    System.out.println(q.size()); // size → 1
    System.out.println(q.isEmpty()); // isEmpty → false
}
}

```

C++ (Using STL queue)

```

#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    q.push(10);           // enqueue
    q.push(20);
    cout << q.front() << endl; // front → 10
    q.pop();              // dequeue
    cout << q.front() << endl; // front → 20
    cout << q.size() << endl;  // size → 1
    cout << q.empty() << endl; // isEmpty → 0 (false)
}

```